CprE 458/558: Real-Time Systems

Overload Handlin in Real-Time Systems

- Imprecise Computations (Part 1)
- (m,k)-firm task model (Part 2)
- Best-Effort Scheduling (Part 3)

How Overloads occur?

- Dynamic workload beyond anticipated
  - Aperiodic tasks and/or dynamically arriving periodic tasks
- Unanticipated faults
  - Frequency and duration of the faults

Imprecise Computational Model

- A way to avoid timing faults during transient overloads and a way to introduce fault-tolerance by graceful degradation is the use of Imprecise Computation (IC) technique.

- The IC model provides scheduling flexibility by trading off result quality to meet task deadlines. A task is divided into a *mandatory* and an *optional part*.

- The mandatory part must be completed before the task's deadline for an acceptable quality of result.

Precise vs Imprecise results

- The optional part, which can be skipped in order to conserve system resources, refines the result.

- A task is said to have produced a **precise** result if it has executed its mandatory as well as optional parts before its deadline;

- otherwise it is said to have produced **imprecise** (i.e., approximate) result when it executes the mandatory part alone.

Monotone vs 0/1 constraint tasks

- There are two types of imprecise computational tasks, namely, **monotone tasks** and **0/1 constraint tasks**.

- A task is *monotone* if the quality of its intermediate result does not decrease as it executes longer.

- An imprecise task with *0/1 constraint* requires the optional part to be either fully executed or not at all.

Applications of Imprecise Computations

- Applications are where one may prefer timely imprecise results to late precise results.
- In image processing, it is often better to have frames of fuzzy images in time than perfect images.
- In radar tracking, it is often better to have estimates of target locations in time than accurate location data too late.

Applications (Contd')

- For example, in a tracking and control system, a transient fault may cause tracking computation to terminate prematurely and produce an approximate result. No recovery action is needed if the result still allows the system to maintain a track of its targets.

- Similarly, as long as the approximate result produced by a control law computation is sufficiently accurate for the controlled system to remain stable, the fault that causes the computation to terminate prematurely can be tolerated.

Error Function & Objective Functions

- Monotone task, Ti: (mi, oi, di)

  Mandatory comp. time (mi), optional comp time (oi), deadline (di)

  – Error ei = F(oi, ki) = oi – ki.

  where ei: Error incurred by task Ti

  ki: optional portion completed

- Minimize the total error
- Minimize the number of optional tasks discarded
  – Shortest processing time first strategy
- Minimize the number of tardy tasks

Algo F (Min Total Error, monotone task, identical weights, optimal, O(n logn))

- Treat all mandatory tasks as optional.
- Use *ED policy* to schedule all the tasks. (St)
- If a feasible schedule is found, precise schedule is obtained, stop.
- Else use ED to schedule mandatory tasks. (Sm)
- If feasible schedule is not found, infeasible schedule, stop.
- Else use Sm as a template, transform St into an optimal schedule that is feasible and minimizes the total error.

Scheduling to Minimize Total Error (for IC tasks with 0/1 constraints)

- The general problem of optimal scheduling of IC tasks with 0/1 constraints is NP-complete.
- Optimal schedule: A schedule in which the number of discarded optional tasks is minimum.
- Special case: Optional tasks have equal comp. time
  - LDF algorithm
    - Same ready time
    - O(n logn) complexity
  - DFS algorithm
    - Arbitrary ready time
    - O(n^2) complexity

Scheduling periodic tasks

- Error-cumulative
  - Tracking and control applications
- Error-non-cumulative
  - Image enhancement and speech processing applications

(m, k) firm real-time tasks

- A periodic task is said to have an (m,k)-firm guarantee if it is adequate to meet the deadlines of m out of k consecutive instances of the task, where $m \leq k$.

- The adaptive QoS management problem
  - Admit the tasks to satisfy at least the (m,k) guarantee
  - Maximize the QoS of admitted tasks beyond the (m,k) property, at run-time, without violating (m,k) property of any of the admitted tasks.

(m,k)-firm deadline model

- A periodic task is said to have an (m,k)-firm guarantee if it is adequate to meet the deadlines of m out of k consecutive instances of the task, where m <= k.

- Periodic task: (pi, ci, mi, ki)
- A flexible method for expressing timing requirements.
- Allows “graceful degradation” during overloads.
- Choose values for m and k such that desired m/k is obtained.
- (1,1)-firm ➔ hard real-time task.
- Apps: Radar tracking, Automobile control

- (m,k) vs. imprecise computation (IC): In (m,k) model instances can be dropped in full; in IC, portion of a instance can be dropped.

Task model and performance index

- Task Model - firm periodic tasks [1,2]

$$T_i =< c_i, p_i, m_i, k_i > \qquad c_i : comp.time \qquad p_i : period$$

- Tasks should meet m_i deadlines for every k_i consecutive instances

- Dynamic failure (Timing failure) is said to occur when (m,k) property is violated for one or more tasks.

- State diagram model – to keep track of temporal history of task execution (M: Meeting deadline, m: missing deadline). This was discussed in the lecture.

MK-RMS Schedulability Check [2]

- Utilization-based MK-RMS-schedulability check (sufficient, but not necessary)

$$mkLoad = \sum_{i=1}^{n} \frac{c_i \cdot m_i}{p_i \cdot k_i}$$

MKLoad <= $n(2^{1/n} - 1)$

- Classification of mandatory and optional instances
  - Instances of task T_i activated at times ap_i is mandatory if

$$a = \left\lfloor \left\lceil \frac{a \cdot m_i}{k_i} \right\rceil \cdot \frac{k_i}{m_i} \right\rfloor \qquad a = 0,1,2,\cdots$$

- Optional instance is assigned the lowest priority
- Mandatory instances are assigned priority as per RMS

References

1. J.W.S. Liu, K.J. Lin, W.K. Shih, A.C. Yu, J.Y.Chung, and W. Zhao, “Algorithms for scheduling imprecise computations,” *IEEE Computer,* vol.24, no.5, pp.58-68, May 1991.

2. P. Ramanathan, “Graceful degradation in real-time control applications using (m,k)-firm guarantee,” In Proc. of Fault-Tolerant Computing Symposium, pp.132-141, 1997.